



CMR UNIVERSITY

Private University Established in Karnataka State by Act No. 45 of 2013

SCHOOL OF ENGINEERING AND TECHNOLOGY

Project Work
On
“EGG CATCHER GAME”
Problem Solving Using Python

Bachelor of Technology
In
COMPUTER SCIENCE AND ENGINEERING
SOET, CMR UNIVERSITY BANGALORE

Submitted By:
BHOOMIKA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CMR University

Off Hennur - Bagalur Main Road, Near Kempegowda International
Airport, Chalahatti, Bengaluru, Karnataka-562149

2025-2026

TABLE OF CONTENTS

S.NO	TITLE
1	ABSTRACT
2	INTRODUCTION
3	PROJECT STATEMENT AND SOLUTIONS
4	IMPLEMENTAION OF CODE
5	SOURCE CODE
6	OUTPUT
7	ALGORITHM
8	PSEUDOCODE
9	FLOWCHART
10	CONCLUSION
11	REFERENCES

ABSTRACT

This report presents the design and implementation of Egg Catcher Master, a 2D desktop arcade game developed in Python using the standard tkinter graphical user interface (GUI) framework. The project demonstrates the practical application of event-driven programming, object movement, dynamic collision detection, and game loop mechanics without relying on heavy third-party game engines.

The game core loop involves catching randomly generated, multi-colored eggs falling from the top of the screen using a user-controlled basket mapped to the keyboard's left and right arrow keys. To enhance engagement and replayability, an exponential difficulty scaling algorithm was implemented; as the player's score increases, the interval delay between animation frames decreases, causing the eggs to accelerate dynamically.

Key technical :

- Efficient asset management utilizing tkinter.Canvas shapes.
 - Infinite color shifting leveraging the itertools.cycle iterator.
 - Precise bounding-box coordinate checking for accurate spatial collision detection.
 - Asynchronous event scheduling via the root.after() method.
-

INTRODUCTION

In the field of software engineering, game development serves as an excellent vehicle for mastering core programming paradigms, such as real-time state management, asynchronous event loops, and graphical user interface (GUI) interactions. While modern, feature-rich frameworks like Pygame or Unity are typically preferred for complex 2D and 3D productions, building a game from scratch using native, lightweight libraries offers deeper insights into the fundamental mechanics of computer graphics and event-driven architectures.

This report details the development of Egg Catcher Master, a 2D desktop arcade game implemented entirely in Python using tkinter, the standard built-in GUI toolkit. The primary objective of this project was to construct a responsive, engaging game loop and physics system utilizing only foundational system libraries, avoiding the overhead of heavy third-party dependencies.

PROJECT STATEMENT AND SOLUTION

Developing a real-time interactive game within a standard, non-gaming GUI framework like Python's tkinter presents several fundamental software engineering challenges. Traditional desktop GUI applications are designed to be passive, waiting silently for a user to click a button or type text before executing a piece of code.

By contrast, an arcade game requires an active environment where the system must constantly update graphics, track physics, and manage game states independently of user interaction.

The core problems this project seeks to address are:

- **The Single-Thread Concurrency Problem:** How to simulate continuously falling objects (the eggs) while simultaneously listening for real-time user keyboard inputs (moving the basket) without freezing the application.
 - **Vector Spatial Collision:** Accurately mapping and calculating overlapping coordinates between two irregular shapes (a flat tkinter canvas oval and an open arc basket) to determine a valid "catch."
 - **Egg Spawning:** `create_egg()` schedules itself to run every 1500 milliseconds, ensuring a steady stream of new objects.
 - **Egg Animation:** `move_eggs()` continuously updates the vertical position of all active eggs. By using `.after(egg_speed, move_eggs)`, the game creates a seamless animation loop that runs in the background, leaving the main thread completely free to instantly register keyboard inputs.
-

IMPLEMENTATION OF CODE

```
MINI PROJECT.py - C:\Users\Bhoomika D\AppData\Local\Programs\Python\Python314\MINI PROJECT.py (3.14.5)
File Edit Format Run Options Window Help
from itertools import cycle
from random import randrange
import tkinter as tk
from tkinter import messagebox

# --- Game Configuration ---
CANVAS_WIDTH = 800
CANVAS_HEIGHT = 400
EGG_SPEED = 500
EGG_INTERVAL = 1500
DIFFICULTY_FACTOR = 0.95

score = 0
lives_remaining = 3
egg_speed = EGG_SPEED

# --- Setup Window and Canvas ---
root = tk.Tk()
root.title("Egg Catcher Master")
root.resizable(False, False)

c = tk.Canvas(root, width=CANVAS_WIDTH, height=CANVAS_HEIGHT,
              background="deep sky blue")

# Ground
c.create_rectangle(
    -5, CANVAS_HEIGHT - 100,
    CANVAS_WIDTH + 5, CANVAS_HEIGHT + 5,
    fill="sea green", width=0
)

# Sun
c.create_oval(-10, -10, 120, 120, fill="orange", width=0)

c.pack()

# --- Game Objects Setup ---
color_cycle = cycle([
    "light blue", "light green",
    "light pink", "peach puff",
    "light yellow"
])

egg_width = 45
egg_height = 55
egg_score = 10
```

```
MINI PROJECT.py - C:\Users\Bhoomika D\AppData\Local\Programs\Python\Python314\MINI PROJECT.py (3.14.5)
File Edit Format Run Options Window Help

basket = c.create_arc(
    CANVAS_WIDTH / 2 - 50,
    CANVAS_HEIGHT - 65,
    CANVAS_WIDTH / 2 + 50,
    CANVAS_HEIGHT - 15,
    start=200,
    extent=140,
    fill="chocolate",
    width=3
)

# UI
score_text = c.create_text(
    10, 10,
    anchor="nw",
    font=("Arial", 16, "bold"),
    fill="dark blue",
    text=f"Score: {score}"
)

lives_text = c.create_text(
    CANVAS_WIDTH - 10,
    10,
    anchor="ne",
    font=("Arial", 16, "bold"),
    fill="red",
    text=f"Lives: {lives_remaining}"
)

eggs = []

def create_egg():
    """Create a new egg at random x position."""
    x = randrange(40, CANVAS_WIDTH - 40)
    y = 40

    egg = c.create_oval(
        x, y,
        x + egg_width,
        y + egg_height,
        fill=next(color_cycle),
        width=2
    )

    eggs.append(egg)
    root.after(EGG_INTERVAL, create_egg)
```

```
MINI PROJECT.py - C:\Users\Bhoomika D\AppData\Local\Programs\Python\Python314\MINI PROJECT.py (3.14.5)
File Edit Format Run Options Window Help

for egg in eggs.copy():
    c.move(egg, 0, 10)

    egg_pos = c.coords(egg)

    if egg_pos:
        # Check if caught
        if egg_caught(egg_pos):
            eggs.remove(egg)
            c.delete(egg)

            score += egg_score
            c.itemconfigure(score_text, text=f"Score: {score}")

            egg_speed = max(50, int(egg_speed * DIFFICULTY_FACTOR))

        # Check if missed
        elif egg_pos[3] > CANVAS_HEIGHT:
            eggs.remove(egg)
            c.delete(egg)

            lives_remaining -= 1
            c.itemconfigure(
                lives_text,
                text=f"Lives: {lives_remaining}"
            )

            if lives_remaining == 0:
                game_over()
                return

    root.after(egg_speed, move_eggs)

def egg_caught(egg_pos):
    """Check collision with basket."""
    basket_pos = c.coords(basket)

    egg_center_x = (egg_pos[0] + egg_pos[2]) / 2
    egg_bottom_y = egg_pos[3]

    return (
        basket_pos[0] < egg_center_x < basket_pos[2]
        and basket_pos[1] < egg_bottom_y < basket_pos[3] + 20
    )

27°C
Mostly cloudy
18:17
07-06-2026
```

```
MINI PROJECT.py - C:\Users\Bhoomika D\AppData\Local\Programs\Python\Python314\MINI PROJECT.py (3.14.5)
File Edit Format Run Options Window Help

def egg_caught(egg_pos):
    """Check collision with basket."""
    basket_pos = c.coords(basket)

    egg_center_x = (egg_pos[0] + egg_pos[2]) / 2
    egg_bottom_y = egg_pos[3]

    return (
        basket_pos[0] < egg_center_x < basket_pos[2]
        and basket_pos[1] < egg_bottom_y < basket_pos[3] + 20
    )

def move_left(event):
    basket_pos = c.coords(basket)

    if basket_pos[0] > 0:
        c.move(basket, -20, 0)

def move_right(event):
    basket_pos = c.coords(basket)

    if basket_pos[2] < CANVAS_WIDTH:
        c.move(basket, 20, 0)

def game_over():
    messagebox.showinfo(
        "Game Over",
        f"Final Score: {score}"
    )
    root.destroy()

# Keyboard Controls
root.bind("<Left>", move_left)
root.bind("<Right>", move_right)

# Start Game
create_egg()
move_eggs()

root.mainloop()

27°C
Mostly cloudy
18:17
07-06-2026
```

SOURCE CODE

```
from itertools import cycle
from random import randrange
import tkinter as tk
from tkinter import messagebox

# --- Game Configuration ---
CANVAS_WIDTH = 800
CANVAS_HEIGHT = 400
EGG_SPEED = 500
EGG_INTERVAL = 1500
DIFFICULTY_FACTOR = 0.95

score = 0
lives_remaining = 3
egg_speed = EGG_SPEED

# --- Setup Window and Canvas ---
root = tk.Tk()
root.title("Egg Catcher Master")
root.resizable(False, False)

c = tk.Canvas(root, width=CANVAS_WIDTH, height=CANVAS_HEIGHT,
               background="deep sky blue")

# Ground
c.create_rectangle(
    -5, CANVAS_HEIGHT - 100,
    CANVAS_WIDTH + 5, CANVAS_HEIGHT + 5,
    fill="sea green", width=0
)
```

```
# Sun
c.create_oval(-10, -10, 120, 120, fill="orange", width=0)

c.pack()

# --- Game Objects Setup ---
color_cycle = cycle([
    "light blue", "light green",
    "light pink", "peach puff",
    "light yellow"
])

egg_width = 45
egg_height = 55
egg_score = 10

# Basket
basket = c.create_arc(
    CANVAS_WIDTH / 2 - 50,
    CANVAS_HEIGHT - 65,
    CANVAS_WIDTH / 2 + 50,
    CANVAS_HEIGHT - 15,
    start=200,
    extent=140,
    fill="chocolate",
    width=3
)

# UI
score_text = c.create_text(
    10, 10,
    anchor="nw",
    font=("Arial", 16, "bold"),
    fill="dark blue",
    text=f"Score: {score}"
)
```

```
lives_text = c.create_text(
    CANVAS_WIDTH - 10,
    10,
    anchor="ne",
    font=("Arial", 16, "bold"),
    fill="red",
    text=f"Lives: {lives_remaining}"
)
```

```
eggs = []
```

```
def create_egg():
    """Create a new egg at random x position."""
    x = randrange(40, CANVAS_WIDTH - 40)
    y = 40
```

```
    egg = c.create_oval(
        x, y,
        x + egg_width,
        y + egg_height,
        fill=next(color_cycle),
        width=2
    )
```

```
    eggs.append(egg)
    root.after(EGG_INTERVAL, create_egg)
```

```
def move_eggs():
    """Move eggs downward."""
    global score, lives_remaining, egg_speed

    for egg in eggs.copy():
        c.move(egg, 0, 10)
        egg_pos = c.coords(egg)
```

```
if egg_pos:
    # Check if caught
    if egg_caught(egg_pos):
        eggs.remove(egg)
        c.delete(egg)

        score += egg_score
        c.itemconfigure(score_text, text=f'Score: {score}')

        egg_speed = max(50, int(egg_speed * DIFFICULTY_FACTOR))

    # Check if missed
    elif egg_pos[3] > CANVAS_HEIGHT:
        eggs.remove(egg)
        c.delete(egg)

        lives_remaining -= 1
        c.itemconfigure(
            lives_text,
            text=f'Lives: {lives_remaining}'
        )

        if lives_remaining == 0:
            game_over()
            return

root.after(egg_speed, move_eggs)
```

```
def egg_caught(egg_pos):
    """Check collision with basket."""
    basket_pos = c.coords(basket)

    egg_center_x = (egg_pos[0] + egg_pos[2]) / 2
    egg_bottom_y = egg_pos[3]
```

```
    return (
        basket_pos[0] < egg_center_x < basket_pos[2]
        and basket_pos[1] < egg_bottom_y < basket_pos[3] + 20
    )
```

```
def move_left(event):
    basket_pos = c.coords(basket)
```

```
    if basket_pos[0] > 0:
        c.move(basket, -20, 0)
```

```
def move_right(event):
    basket_pos = c.coords(basket)
```

```
    if basket_pos[2] < CANVAS_WIDTH:
        c.move(basket, 20, 0)
```

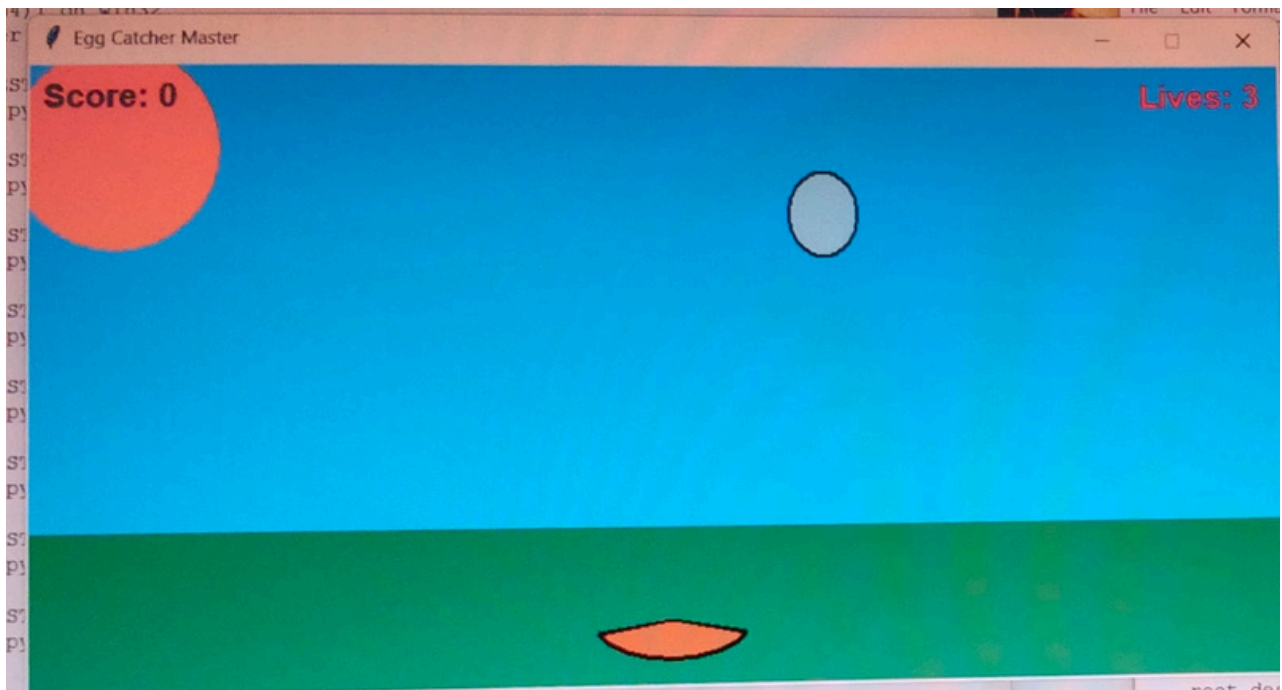
```
def game_over():
    messagebox.showinfo(
        "Game Over",
        f"Final Score: {score}"
    )
    root.destroy()
```

```
# Keyboard Controls
root.bind("<Left>", move_left)
root.bind("<Right>", move_right)
```

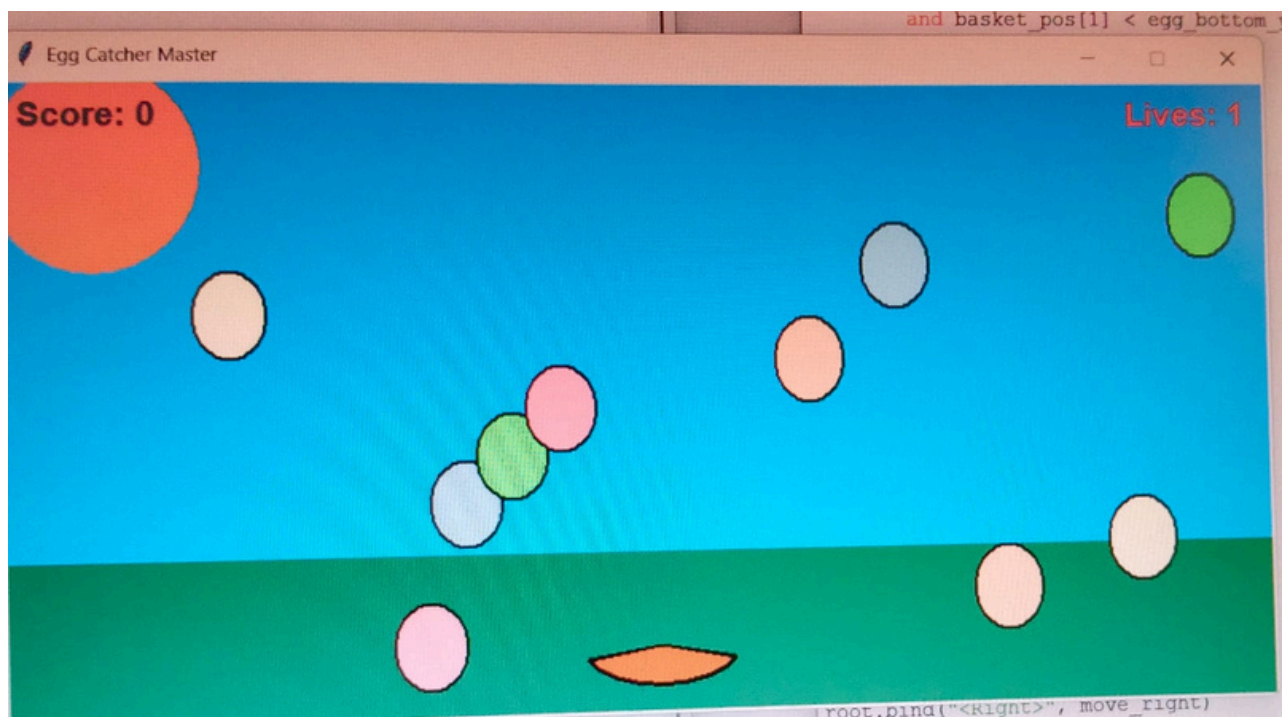
```
# Start Game
create_egg()
move_eggs()
```

```
root.mainloop()
```

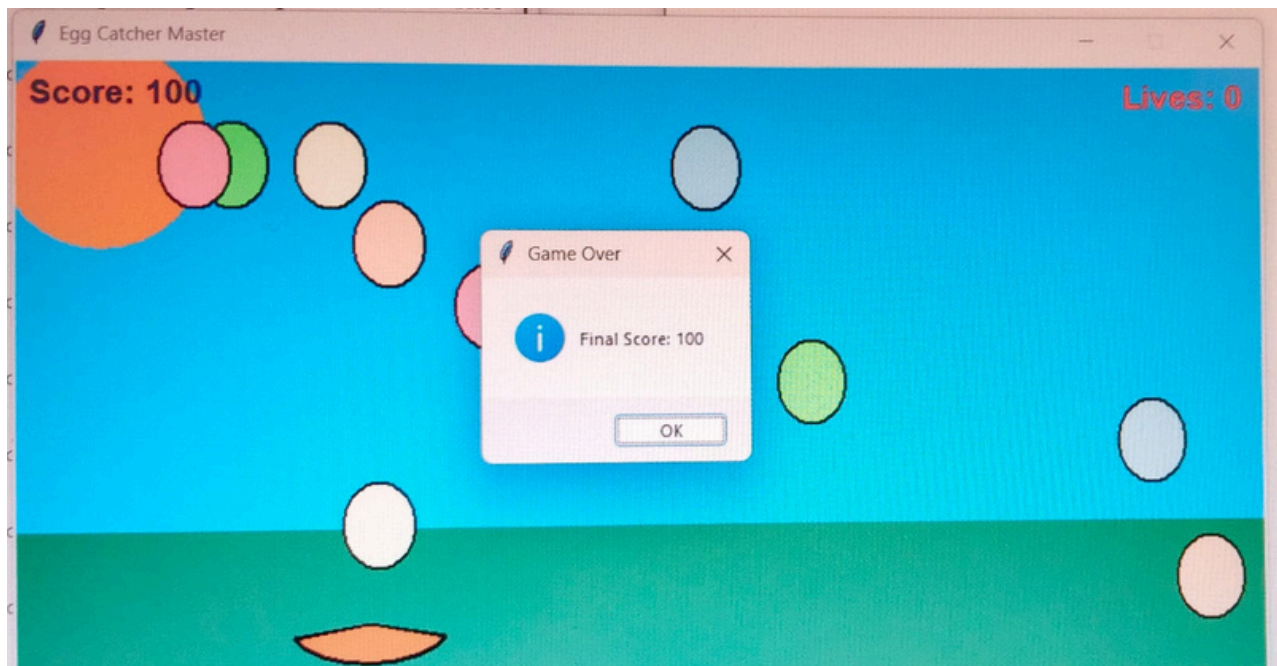
OUTPUT



GAME START



GAME RUNNING



GAME END

ALGORITHM

1. Initialization Stage

- Initialize Canvas: Create an 800 \times 400 window with a ground plane, a sun graphic, and a basket positioned at the bottom center.
- Initialize State Variables: * $\text{score} = 0$
 1. $\text{lives_remaining} = 3$
 2. $\text{egg_speed} = 500$ (milliseconds per frame update)
 3. $\text{eggs} = []$ (an empty list tracking active items)

2. Async Event: User Input (Basket Movement)

- If Left Arrow: Check if the basket is away from the left wall ($\text{basket_x1} > 0$). If yes, shift it left by 20 pixels.
- If Right Arrow: Check if the basket is away from the right wall ($\text{basket_x2} < \text{CANVAS_WIDTH}$). If yes, shift it right by 20 pixels

3. Loop A: Egg Spawning (Runs every 1.5 seconds)

1. Pick a random horizontal position X between 40 and 760.
 2. Select the next color from a continuous cycle.
 3. Draw a new oval asset at $(X, 40)$ and add its unique ID to the eggs list.
 4. Schedule the next execution of Loop A in 1500 milliseconds
-

-
4. Loop B: Physics & Game Loop (Runs every egg_speed:)
 1. Move: Translate the egg asset downwards by 10 vertical pixels.
 2. Fetch Position: Get the current bounding box coordinates [x1, y1, x2, y2] of the egg.
 3. Check Condition - Caught: * Is the egg's horizontal center between the basket's left and right rims AND is its bottom edge within the basket opening?
 4. Repeat: If lives are still above 0, schedule the next execution of Loop B to run after the updated egg_speed interval.

5. Game Over Routine

Halt all movement updates.

Display a popup box showing the final achieved score.

Completely destroy the game window environment (root.destroy()).

PSEUDOCODE

// --- GAME LOOP (Runs every 'egg_speed' milliseconds) ---

PROCEDURE move_eggs()

FOR EACH egg IN eggs DO

MoveEggDown(egg, 10)

IF EggIsCaught(egg) THEN

Remove(egg)

score = score + 10

egg_speed = egg_speed * 0.95 // Increase speed

ELSE IF EggHitGround(egg) THEN

Remove(egg)

lives = lives - 1

IF lives == 0 THEN

EndGame()

END IF

END IF

END FOR

END PROCEDURE

// --- COLLISION LOGIC ---

FUNCTION EggIsCaught(egg)

 egg_center_x = (egg.left + egg.right) / 2

 IF (egg_center_x is between basket.left and basket.right) AND
 (egg.bottom is inside basket.rim) THEN

 RETURN TRUE

 ELSE

 RETURN FALSE

 END IF

END FUNCTION

// --- SPAWNER (Runs every 1.5 seconds) ---

PROCEDURE create_egg()

 spawn_x = RandomNumber(40, CANVAS_WIDTH - 40)

 new_egg = CreateEggObject(at: spawn_x, y: 40)

 Add new_egg to eggs

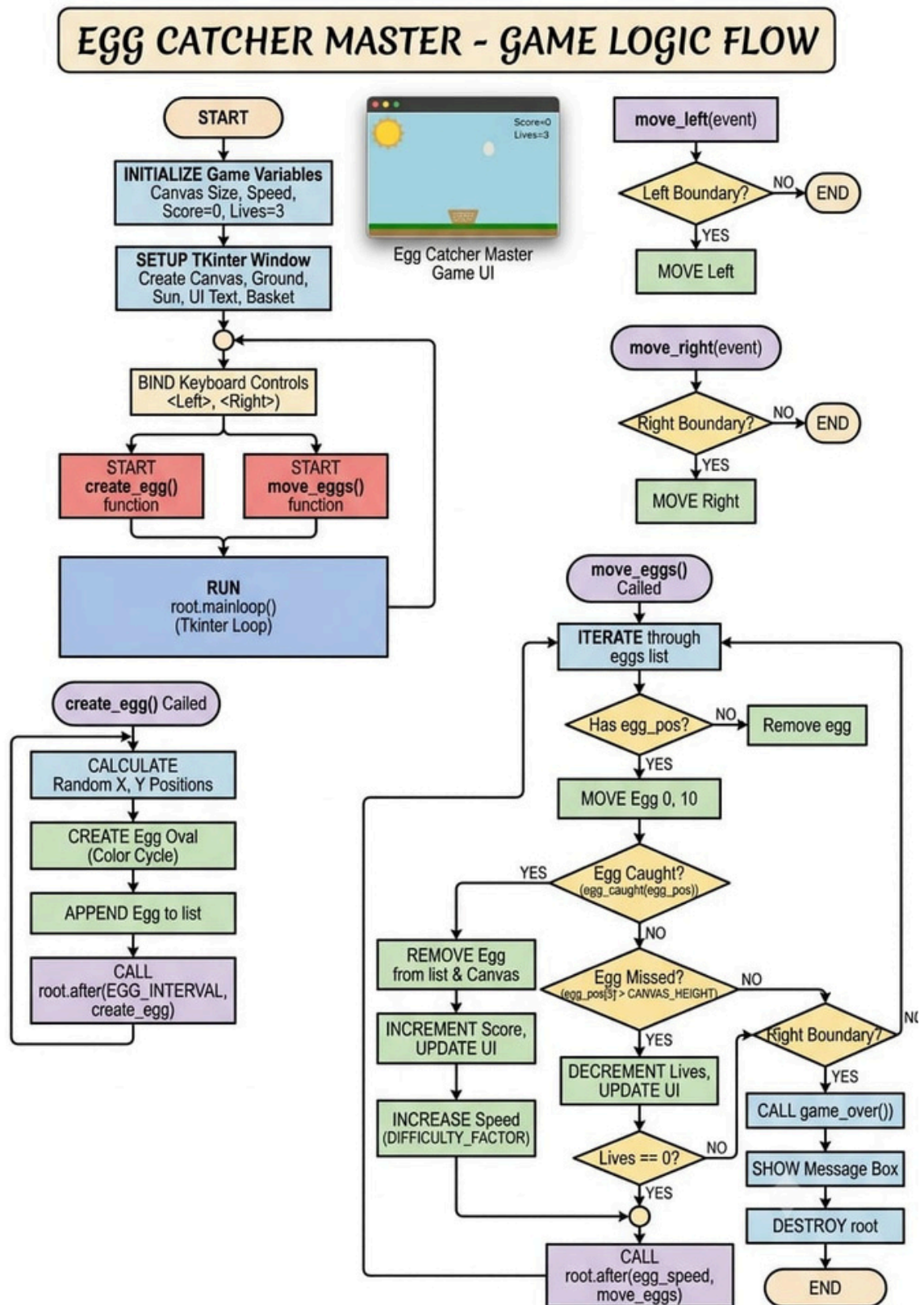
END PROCEDURE

// --- CONTROLS ---

PROCEDURE move_left() IF basket.left > 0 THEN
ShiftBasket(-20)

PROCEDURE move_right() IF basket.right < CANVAS_WIDTH
THEN ShiftBasket(20)

FLOWCHART



CONCLUSION

The Egg Catcher Master project successfully implements a classic 2D arcade game using a clean, event-driven architecture. By shifting away from traditional, heavy game engines and utilizing Python's native Tkinter library, the code demonstrates how asynchronous timing loops (`root.after`) can handle complex tasks like real-time collision detection, physics updates, and user input tracking simultaneously without lagging the interface.

Ultimately, this code serves as an excellent foundational blueprint for software design. It perfectly highlights how basic geometry, conditional logic, and asynchronous execution come together to transform a simple script into an interactive, scaling, and responsive user experience.

REFERENCES

- Python 3 Documentation. Retrieved from <https://docs.python.org/3/>
 - tkinter — Python Interface to Tcl/Tk. Retrieved from <https://docs.python.org/3/library/tkinter.html>
 - Python GUI Programming With Tkinter. Retrieved from <https://realpython.com/python-gui-tkinter/>
 - GeeksforGeeks. GUI Programming with Tkinter. Retrieved from <https://www.geeksforgeeks.org/python-gui-tkinter/>
-

